Bézier Curve: Quadratic (2-D)

CairoMakie Visualisation

Code and Plot Explanation for Julia Beginners

`11-bezier-quadratic-makie.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains `11-bezier-quadratic-makie.jl`. It is the CairoMakie counterpart to `11-bezier-quadratic-plots.jl`. The curve, control points, and all computed values are identical — only the plotting library and its API differ. This document is fully self-contained but deliberately highlights the differences from Plots.jl so you can read the two side by side.

Contents

1 The Plot **2**

2 Reading the Plot **2**
2.1 Visual differences from the Plots.jl version 2

3 What is CairoMakie? **3**

4 Type Map **4**

5 Data, Constants, and Math Functions **4**

6 `draw_and_save` — The Makie Drawing API **4**
6.1 Step 1: Create the Figure 5
6.2 Step 2: Create the Axis 5
6.3 Step 3: Draw the Curve — `lines!` 5
6.4 Step 4: Draw the Control Polygon — Two Separate Calls 6
6.5 Step 5: Annotate Control Points — `text!` 6

6.6 Step 6: Highlighted Points and Labels 7
6.7 Step 7: Equation Text 7
6.8 Step 8: The Legend — `axislegend` 7
6.9 Step 9: Save the Figure 8

7 API Comparison: Plots.jl vs CairoMakie 9

8 Entry Point 9

1. The Plot

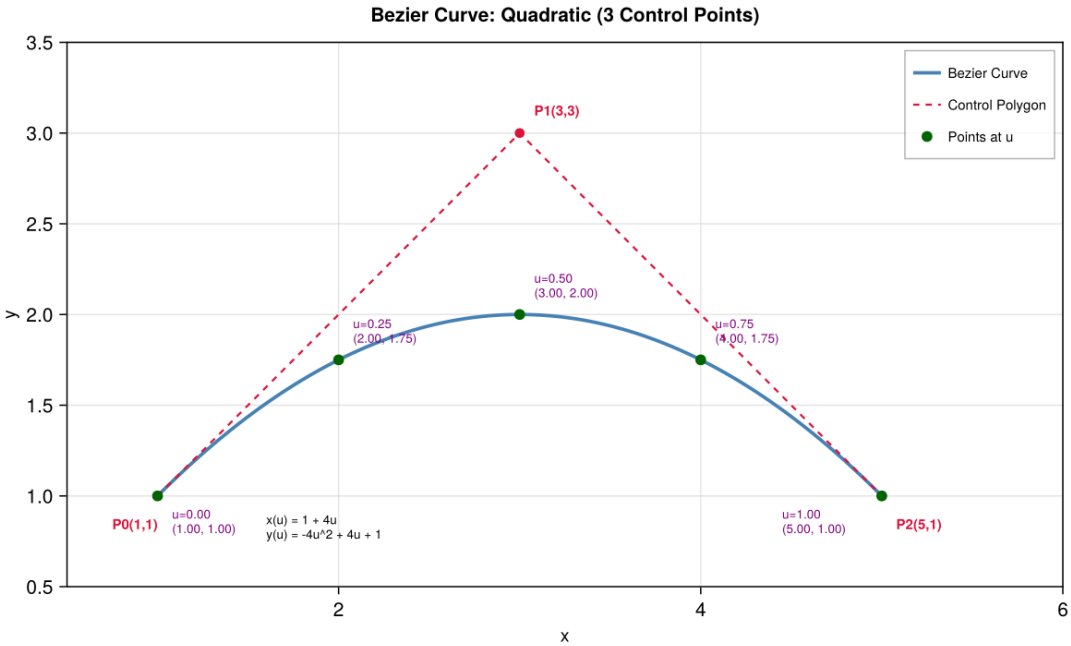


Figure 1: Quadratic Bézier curve with 3 control points, plotted with CairoMakie.

2. Reading the Plot

The content is identical to the Plots.jl version: the same blue curve, red dashed control polygon, green highlighted dots, purple annotations, and equation text. The geometric meaning is unchanged.

2.1. Visual differences from the Plots.jl version

Looking at the two plots side by side, several visual differences are immediately visible:

Element	Plots.jl	CairoMakie
Frame	Four-sided box	Open axes (no right/top border)
Title font	Normal weight	**Bold**
Background	White with grid	White with lighter grid lines
Control polygon markers	Medium circles	Slightly larger circles
Resolution	800×600 px	800×600 × 1.5 = 1200×900 px
Legend	Inside plot area	Separate legend box, top-right

These differences come entirely from the two libraries' default styling and rendering engines, not from any difference in the underlying curve data.

> **CairoMakie vs Plots.jl rendering.** Plots.jl uses the GR backend by default, which renders to screen at 1:1 pixel ratio. CairoMakie uses Cairo, a vector graphics library, and the `px_per_unit = 1.5` argument in `save()` produces a higher-resolution PNG: $800 \times 1.5 = 1200$ pixels wide instead of 800. This is why the CairoMakie output looks sharper.

3. What is CairoMakie?

Makie is Julia's other major plotting framework. Understanding how it differs from Plots.jl explains why the code looks so different even though it draws the same thing.

Makie.jl is a *native Julia* plotting framework built from the ground up for performance and flexibility. It does not wrap another library — it renders directly using its own scene graph. There are three backends:

Backend	Use case
`CairoMakie`	Static output to PNG, SVG, PDF. Uses the Cairo library. Good for publication-quality figures.
`GLMakie`	Interactive plots in a GPU-accelerated window. Good for real-time updates and 3-D rotation.
`WGLMakie`	Interactive plots in a web browser via WebGL. Good for Jupyter notebooks and web deployment.

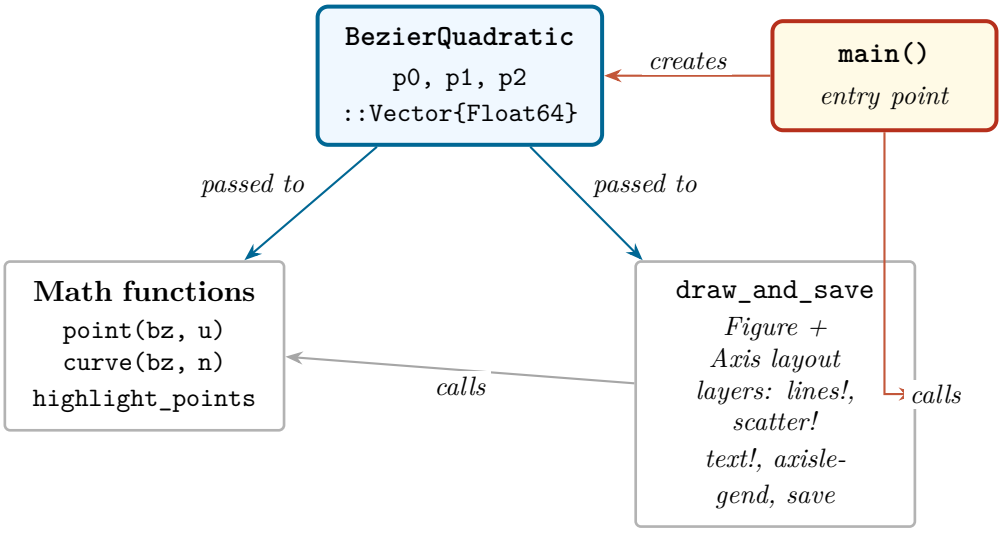
This script uses `CairoMakie` because it saves to PNG without needing a display server.

The key architectural difference from Plots.jl: In Plots.jl, `plot()` creates and returns a plot object that is both the figure and the axes combined. In Makie, these are always separate: `Figure` is the canvas, `Axis` is a coordinate system placed inside the figure. This separation gives much finer control over layout.

Install once with:

```
import Pkg; Pkg.add("CairoMakie")
```

4. Type Map



The structure is identical to the Plots.jl version. Only the internals of `draw_and_save` changed.

5. Data, Constants, and Math Functions

`CONTROL_POINTS`, `U_HIGHLIGHT`, `U_OFFSETS`, the colour constants, `BezierQuadratic` (struct and constructor), `curve`, and `highlight_points` are **identical** to the Plots.jl version. Please refer to `11-bezier-quadratic-plots-explained` for full explanations of those components.

One small difference: `point()` no longer calls a separate `basis()` function. The Bernstein weights are computed inline:

```
function point(bz::BezierQuadratic, u::Float64)
    b0 = (1 - u)^2
    b1 = 2 * u * (1 - u)
    b2 = u^2
    b0 .* bz.p0 .+ b1 .* bz.p1 .+ b2 .* bz.p2
end
```

This is functionally identical to the Plots.jl version where `b0, b1, b2 = basis(u)` was called first. The computation and result are the same; the `basis` helper is simply not factored out as a separate function here.

All data tables (`cpts`, `cp`, `hpts`, `pt` values) are also identical to the Plots.jl version since the math is unchanged.

6. `draw_and_save` — The Makie Drawing API

This is where the two scripts diverge completely. Makie separates figure, axes, and drawing into distinct steps. Plots.jl collapsed all of these into a single `plot()` call.

6.1. Step 1: Create the Figure

```
fig = Figure(size = (800, 600), backgroundcolor = :white)
```

`Figure` is the top-level canvas. It has a size and a background colour, but it does not yet have any coordinate system or data. Think of it as a blank sheet of paper.

> **Plots.jl:** `plt = plot(size=(800,600), background=:white, ...)` — one call creates figure, axes, and layout all at once.
>
> **CairoMakie:** `fig = Figure(...)` creates only the canvas. The axes are created separately in the next step.

6.2. Step 2: Create the Axis

```
ax = Axis(fig[1, 1],
    title           = "Bezier Curve: Quadratic (3 Control Points)",
    titlesize       = 14,
    titlegap        = 12,
    xlabel          = "x",
    ylabel          = "y",
    xlabelsize      = 13,
    ylabelsize      = 13,
    limits          = (0.5, 6.0, 0.5, 3.5),
    aspect          = DataAspect(),
    xgridcolor      = (:lightgray, 0.6),
    ygridcolor      = (:lightgray, 0.6),
    backgroundcolor = :white,
)
```

`fig[1, 1]`: Makie uses a **grid layout** to place axes inside a figure. `fig[1, 1]` means row 1, column 1 of the figure grid. A figure can hold multiple axes in different grid positions (e.g. `fig[1,1]`, `fig[1,2]`, `fig[2,1]` for a 2×2 grid of subplots). Here there is only one axis, placed at position $(1, 1)$.

`limits = (xmin, xmax, ymin, ymax)`: Sets the visible range of the axes. In Plots.jl this was two separate keyword arguments `xlims` and `ylims`. Makie combines them into one 4-tuple.

`aspect = DataAspect()`: Forces equal scaling of x and y axes, just like `aspect_ratio = :equal` in Plots.jl. `DataAspect()` is a Makie type that means "use the data coordinate ratio, not the pixel ratio."

`xgridcolor = (:lightgray, 0.6)`: Makie accepts colours as **(name, alpha)** tuples. `(:lightgray, 0.6)` means light gray at 60% opacity. In Plots.jl the equivalent was separate `gridcolor` and `gridalpha` arguments.

The grid layout system:

A single plot looks like this in memory:

```
fig = Figure(...)          # canvas (blank paper)
ax  = Axis(fig[1,1], ...)  # axes at grid position (1,1)
```

A two-column subplot layout would be:

```
ax1 = Axis(fig[1,1], ...)  # left axes
ax2 = Axis(fig[1,2], ...)  # right axes
```

This grid system is why Makie is preferred for complex multi-panel figures.

6.3. Step 3: Draw the Curve — `lines!`

```
cpts = curve(bz)
lines!(ax, cpts[:,1], cpts[:,2],
    color     = COLOR_CURVE,
    linewidth = 2.5,
    label     = "Bezier Curve",
)
```

In Makie, `lines!` is the explicit function for drawing a connected line through a series of points. The first argument is the **axis** `ax`, not the figure or a plot object.

Plots.jl: `plot!(plt, xs, ys, ...)` — `plot!` is a generic function that draws a line by default; the type of plot is inferred from options.

CairoMakie: `lines!(ax, xs, ys, ...)` — the function name explicitly states what is drawn. Makie has separate functions: `lines!` for lines, `scatter!` for points, `band!` for filled regions, etc. There is no generic `plot!` that guesses the type.

The `cpts` matrix is identical to the Plots.jl version: 200×2, with `cpts[:,1]` the x column and `cpts[:,2]` the y column.

6.4. Step 4: Draw the Control Polygon — Two Separate Calls

```
cp = vcat(bz.p0', bz.p1', bz.p2')
lines!(ax, cp[:,1], cp[:,2],
    color     = COLOR_POLYGON,
    linestyle = :dash,
    linewidth = 1.5,
    label     = "Control Polygon",
)
scatter!(ax, cp[:,1], cp[:,2],
    color       = COLOR_POLYGON,
    markersize  = 10,
    strokewidth = 0,
)
```

This is the most visible API difference from Plots.jl.

In Plots.jl, one `plot!` call with `marker = :circle` drew both the dashed line *and* the circles at the control points.

In Makie, lines and scatter markers are always separate drawing operations. `lines!` draws the dashed connecting line. `scatter!` draws the red circles at P_0, P_1, P_2. The second `scatter!` call has no `label` argument — it should not appear in the legend separately because it is part of the same "Control Polygon" visual element.

Content of `cp` (same as Plots.jl version):

$$\texttt{cp} = \begin{bmatrix} 1.0 & 1.0 \\ 3.0 & 3.0 \\ 5.0 & 1.0 \end{bmatrix}$$

`cp[:,1] = [1.0, 3.0, 5.0]` — x coords of control points

`cp[:,2] = [1.0, 3.0, 1.0]` — y coords of control points

6.5. Step 5: Annotate Control Points — `text!`

```
offsets_ctrl = [(-0.25, -0.20), (0.08, 0.08), (0.08, -0.20)]
names_ctrl   = ["P0(1,1)", "P1(3,3)", "P2(5,1)"]
for (i, (name, (dx, dy))) in enumerate(zip(names_ctrl, offsets_ctrl))
    text!(ax, cp[i,1]+dx, cp[i,2]+dy,
        text     = name,
        color    = COLOR_CTRL,
        fontsize = 10,
        font     = :bold,
    )
end
```

`text!` vs `annotate!`: Makie uses `text!` for placing labels. The text string is passed as a keyword argument `text = name`, not as a positional argument.

`enumerate(zip(...))` with index `i`: The loop uses `enumerate` to get both the index `i` and the `(name, (dx,dy))` tuple simultaneously. This is because Makie's `text!` needs the coordinates in the form `cp[i,1]` and `cp[i,2]` — indexing by row number — rather than `eachrow(cp)` as in the Plots.jl version.

Plots.jl used `eachrow(cp)` to iterate over row vectors, then accessed `pt[1]` and `pt[2]`.

CairoMakie uses `enumerate(zip(...))` to get the row index `i`, then accesses `cp[i,1]` and `cp[i,2]` directly.

Both achieve the same result. The Makie version is slightly more explicit about the index.

6.6. Step 6: Highlighted Points and Labels

```
hpts = highlight_points(bz, u_highlight)
scatter!(ax, hpts[:,1], hpts[:,2],
    color       = COLOR_POINT,
    markersize  = 11,
    strokewidth = 0,
    label       = "Points at u",
)
```

`strokewidth = 0` in Makie is the equivalent of `markerstrokewidth = 0` in Plots.jl — both remove the border ring around each marker.

Content of `hpts` (same as Plots.jl version):

$$\texttt{hpts} = \begin{bmatrix} 1.00 & 1.00 \\ 2.00 & 1.75 \\ 3.00 & 2.00 \\ 4.00 & 1.75 \\ 5.00 & 1.00 \end{bmatrix}$$

```
for u in u_highlight
    pt     = point(bz, u)
    x, y   = pt[1], pt[2]
    dx, dy = get(U_OFFSETS, u, (0.08, 0.08))
    text!(ax, x+dx, y+dy,
        text     = @sprintf("u=%.2f\n(%.2f, %.2f)", u, x, y),
        color    = COLOR_LABEL,
        fontsize = 9,
    )
end
```

The loop logic is identical to Plots.jl. The only difference is using `text!(ax, x, y, text=...)` instead of `annotate!(plt, x, y, text(...))`.

Value of `pt` for each iteration (same as Plots.jl):

u	`pt[1]`	`pt[2]`	Label text	Offset
0.00	1.00	1.00	`u=0.00\n(1.00, 1.00)`	$(+0.08, -0.22)$
0.25	2.00	1.75	`u=0.25\n(2.00, 1.75)`	$(+0.08, +0.08)$
0.50	3.00	2.00	`u=0.50\n(3.00, 2.00)`	$(+0.08, +0.08)$
0.75	4.00	1.75	`u=0.75\n(4.00, 1.75)`	$(+0.08, +0.08)$
1.00	5.00	1.00	`u=1.00\n(5.00, 1.00)`	$(-0.55, -0.22)$

6.7. Step 7: Equation Text

```
text!(ax, 1.6, 0.75,
    text     = "x(u) = 1 + 4u\ny(u) = -4u^2 + 4u + 1",
    color    = :black,
    fontsize = 9,
)
```

Identical in purpose to the Plots.jl version. The position `(1.6, 0.75)` is in data coordinates. The
`n` produces a line break between the two equations.

6.8. Step 8: The Legend — `axislegend`

```
axislegend(ax,
    position        = :rt,
    labelsize       = 10,
    framecolor      = (:black, 0.3),
    backgroundcolor = (:white, 0.8),
)
```

In Plots.jl, the legend is configured as part of the initial `plot()` call with `legend = :topright`.

In Makie, the legend is added as a **separate step** after all the drawing is done, by calling `axislegend`. It reads the `label` strings from the previous `lines!` and `scatter!` calls automatically — only elements that were given a `label` appear in the legend. The control polygon scatter markers had no `label`, so they are correctly excluded.

`position = :rt` means top-right (`:rt` = right-top in Makie's convention).

Colour with alpha: `(:black, 0.3)` means black at 30% opacity — a semi-transparent frame around the legend box. `(:white, 0.8)` means white at 80% opacity — a slightly transparent white background so the legend does not completely hide the plot behind it.

6.9. Step 9: Save the Figure

```
save(filename, fig, px_per_unit = 1.5)
println("Saved: $filename")
display(fig)
println("Press Enter to close...")
readline()
```

`save(filename, fig, px_per_unit=1.5)`: Saves the Makie figure to a file. The filename extension determines the format (`.png`, `.svg`, `.pdf` are all supported).

`px_per_unit = 1.5` is a resolution multiplier. Makie measures figure size in *logical units* (essentially CSS pixels), and this argument scales them to actual pixels in the saved file. With `size = (800, 600)` and `px_per_unit = 1.5`, the saved PNG is $800 \times 1.5 = 1200$ pixels wide and $600 \times 1.5 = 900$ pixels tall. This produces a sharper image than Plots.jl's default 1:1.

> **Plots.jl:** `savefig(plt, filename)` — saves at the pixel size specified in `size`.
>
> **CairoMakie:** `save(filename, fig, px_per_unit=1.5)` — saves at `size` × `px_per_unit` resolution. The higher resolution is visible in the sharper edges and text in the Makie output PNG.

7. API Comparison: Plots.jl vs CairoMakie

A side-by-side reference for the key operations performed in these two scripts.

Operation	Plots.jl	CairoMakie
Create canvas	`plt = plot(...)`	`fig = Figure(...)`
Create axes	(combined with canvas)	`ax = Axis(fig[1,1], ...)`
Draw line	`plot!(plt, xs, ys)`	`lines!(ax, xs, ys)`
Draw scatter	`scatter!(plt, xs, ys)`	`scatter!(ax, xs, ys)`
Line + markers	One `plot!` with `marker=`	Two calls: `lines!` + `scatter!`
Axis limits	`xlims=(...), ylims=(...)`	`limits=(xmin,xmax,ymin,ymax)`
Equal aspect	`aspect_ratio=:equal`	`aspect=DataAspect()`
Grid colour+alpha	`gridcolor=, gridalpha=`	`xgridcolor=(:color, alpha)`
Text annotation	`annotate!(plt, x, y, text(...))`	`text!(ax, x, y, text=...)`
Legend	`legend=:topright` in `plot()`	`axislegend(ax, position=:rt)`
Save to file	`savefig(plt, filename)`	`save(filename, fig, px_per_unit=)`

8. Entry Point

```julia
function main()
    bz = BezierQuadratic(CONTROL_POINTS)
    draw_and_save(bz, U_HIGHLIGHT, "11-bezier-quadratic-makie.png")
end

main()
```

Identical structure to the Plots.jl version. The only difference is the output filename.

How to run:

```julia
julia 11-bezier-quadratic-makie.jl
```

The script saves `11-bezier-quadratic-makie.png` and opens a display window. Press Enter in the terminal to close and exit.

jayadi | Code and explanations AI-generated